

RAJALAKSHMI ENGINEERING COLLEGE
Department of Computer Science and Engineering
Academic Year: 2025 – 2026

MINI PROJECT REPORT

On

**Pothole and Road Crack Detection
for Smart Road Infrastructure**

Individual Report — Algorithm D: Laplacian + Morphological Skeleton Detector

Submitted by

HARESH R

Roll No: 230701097

Section: CSE-B

Email: 230701097@rajalakshmi.edu.in

Date of Submission: May 2026

TABLE OF CONTENTS

1.	Introduction	3
	1.1 Background and Motivation	3
	1.2 Problem Statement	3
	1.3 Objectives	4
	1.4 Scope of the Report	4
2.	Literature Review	5
	2.1 Classical Edge and Morphological Methods	5
	2.2 Deep Learning Approaches	5
	2.3 Gap Identified	6
3.	Dataset Description	6
	3.1 Real Road Image Dataset	6
	3.2 Annotation and Patch Classification	6
	3.3 Data Split	7
4.	Preprocessing Methodology	7
	4.1 Resizing (INTER_AREA)	7
	4.2 Gaussian Denoising	8
	4.3 CLAHE Contrast Enhancement	8
	4.4 Grayscale Conversion	8
	4.5 Gaussian Image Pyramid	9
	4.6 Before vs After Visualisation	9
5.	Algorithm: Laplacian + Morphological Skeleton	10
	5.1 Algorithm Overview	10
	5.2 Stage 1 — Laplacian Edge Detection (Pothole)	10
	5.3 Stage 2 — Morphological Skeleton (Crack)	11
	5.4 Severity Scoring	12
	5.5 Streamlit Frontend Architecture	12
	5.6 Complete Algorithm Flow	13
6.	Results and Analysis	14
	6.1 Quantitative Results	14
	6.2 Effect of Preprocessing	14
	6.3 Severity Heatmap Analysis	15
	6.4 Sample Detection Outputs	15
	6.5 Live Detection Interface	15
7.	Discussion	16
8.	Conclusion and Future Work	17
	References	18
	Appendix A — Source Code	19

Appendix B — Sample Output Screenshots

22

LIST OF TABLES AND FIGURES

Tables

- Table 1: Road Defect Image Dataset Summary
- Table 2: Dataset Split Statistics
- Table 3: Laplacian Filter Parameter Configuration
- Table 4: Morphological Skeleton Configuration
- Table 5: Streamlit Frontend Module Configuration
- Table 6: Quantitative Detection Results per Defect Class
- Table 7: Preprocessing Ablation Study

Figures

- Figure 1: Dataset sample distribution bar chart
- Figure 2: Laplacian response visualisation
- Figure 3: Before vs After preprocessing (3-stage comparison)
- Figure 4: Algorithm architecture flowchart
- Figure 5: Severity heatmap sample output
- Figure 6: Defect mask and annotated detection overlay
- Figure 7: Streamlit UI — Results tab
- Figure 8: Streamlit UI — Pipeline Steps tab

ABSTRACT

This project presents the design, implementation, and evaluation of a Real-Time Pothole and Road Crack Detection System that fuses two complementary classical computer vision algorithms: a Laplacian second-order differential operator for pothole detection and a Morphological Skeleton (Zhang-Suen thinning) pipeline for crack detection, wrapped in a fully interactive Streamlit web application for smart road infrastructure monitoring.

The preprocessing pipeline applies five image processing operations: INTER_AREA resizing, Gaussian denoising, CLAHE contrast enhancement on the L^* luminance channel, grayscale conversion, and Gaussian image pyramid construction for multi-scale analysis. These operations collectively improve detection robustness under the varied lighting, texture, and resolution conditions common in real-world road imagery captured from dashcams, drones, and municipal inspection cameras.

The Laplacian stage applies a second-order differential filter (configurable kernel size 1–7) to the CLAHE-enhanced grayscale frame, producing a magnitude map that fires at abrupt surface depth discontinuities characteristic of pothole rims. Morphological closing merges fragmented edge responses into contiguous blob regions. The Morphological Skeleton stage applies adaptive thresholding to extract dark linear features, then applies the Zhang-Suen iterative thinning algorithm to reduce crack regions to their topological centrelines, enabling precise crack length and orientation measurement.

Both detection streams are combined into a unified defect mask. A severity score (0–100 index) is computed from the weighted defect area fractions and mapped to four actionable severity labels (Low / Medium / High / Critical). A Gaussian-smoothed severity heatmap is generated for spatial risk visualisation. The Streamlit frontend provides real-time parameter tuning, a step-by-step pipeline viewer, quantitative analysis charts, and an exportable Markdown maintenance report.

The complete system requires no GPU, no deep learning framework, and no annotated training dataset. It runs on a standard CPU using only pip-installable packages (OpenCV, scikit-image, scipy, streamlit) and can process a 640×480 road image in under 200 milliseconds.

1. INTRODUCTION

1.1 Background and Motivation

Road infrastructure degradation is a universal challenge for municipal governments and transportation authorities. Potholes and pavement cracks are the two most prevalent forms of road surface damage, arising from cyclic traffic loading, thermal expansion and contraction, subsurface water infiltration, and material fatigue. Undetected and unrepaired defects progressively worsen under continued traffic, leading to vehicle suspension damage, tyre blowouts, and in severe cases, loss of vehicle control at speed.

Traditional road condition surveys rely on trained inspectors walking or driving survey vehicles along road networks, visually assessing surface condition and manually logging defects on paper or in geographic information systems. This process is inherently slow, labour-intensive, subjective, and expensive. A single inspector can survey only a few kilometres of road per day, making comprehensive network-wide assessment impractical for large urban and highway networks.

Computer vision offers a compelling automated alternative. A camera mounted on a standard vehicle, drone, or fixed roadside infrastructure can continuously capture road surface imagery at low cost. Image processing algorithms can then analyse this imagery to detect, classify, and geo-locate defects in real time, enabling authorities to prioritise and schedule maintenance before conditions deteriorate to a safety-critical level. This project implements such a system using classical image processing algorithms that require no GPU, no deep learning infrastructure, and no labelled training data — making it immediately deployable on low-cost embedded hardware.

1.2 Problem Statement

Real-time road defect detection requires a system that can distinguish two fundamentally different defect geometries from the same camera frame within a time budget compatible with continuous video processing. Potholes are localised, roughly circular or elliptical surface depressions characterised by abrupt changes in surface depth and associated shadow patterns visible in greyscale imagery. Road cracks are elongated, branching linear features with very small width (often 1–5 pixels at typical camera resolutions) that form complex dendritic networks across the pavement surface.

A single algorithm is ill-suited to detect both defect types simultaneously. Edge detection alone produces fragmented responses that fail to fill pothole blobs. Thresholding alone misses the topology of crack networks. This project addresses the problem: how to design and implement a dual-stream classical image processing pipeline that simultaneously detects both pothole blobs and crack centrelines from the same input image, produces an interpretable severity score, and presents results through an interactive web interface without requiring any GPU hardware, deep learning framework, or annotated training dataset?

1.3 Objectives

- To design a complete five-stage preprocessing pipeline (INTER_AREA resizing, Gaussian denoising, CLAHE enhancement, grayscale conversion, Gaussian image pyramid) tailored to road surface imagery captured under uncontrolled lighting conditions.
- To implement the Laplacian second-order differential filter pipeline for pothole detection, including magnitude thresholding, morphological closing, and connected component filtering by area and aspect ratio.

- To implement the Morphological Skeleton (Zhang-Suen thinning) pipeline for crack detection, including adaptive thresholding, morphological opening, skeleton extraction, dilation, and elongation-based region filtering.
- To fuse both detection streams into a unified defect mask and compute an interpretable severity score (0–100 index) mapped to actionable maintenance priority labels.
- To generate a Gaussian-smoothed severity heatmap for spatial risk communication to maintenance planners.
- To develop a complete interactive Streamlit web application with real-time parameter tuning, step-by-step pipeline visualisation, quantitative analysis charts, and exportable maintenance reports.
- To evaluate the system on synthetic and real road images, measuring processing speed, defect area estimation accuracy, and severity classification consistency.

1.4 Scope of the Report

This report covers the complete end-to-end pipeline from image acquisition through preprocessing, Laplacian-based pothole detection, morphological skeleton-based crack detection, severity scoring, heatmap generation, and Streamlit frontend deployment. The detection system is designed for CPU-only deployment using Python 3.x with OpenCV, scikit-image, scipy, matplotlib, pandas, and streamlit. No deep learning training framework (TensorFlow, PyTorch) is used. The system accepts input from uploaded images, webcam snapshots, or built-in synthetic road scenes, and outputs annotated images, severity scores, crack region statistics, and downloadable maintenance reports.

2. LITERATURE REVIEW

2.1 Classical Edge and Morphological Methods

Early automated road defect detection research applied classical image processing operators to greyscale road imagery. Cracks were frequently targeted using edge detectors (Sobel, Canny) followed by morphological operations to connect fragmented edge responses. Koch and Brilakis (2011) [1] demonstrated that the Otsu thresholding followed by morphological thinning could reliably extract crack skeletons from controlled-illumination pavement photographs, achieving 78% detection rate on a benchmark of 200 highway crack images.

Pothole detection through image analysis was systematically studied by Mednis et al. (2011) [2] using vibration sensors, and subsequently by Eriksson et al. [3] using accelerometers mounted in vehicles. Image-based pothole detection using depth cameras was explored by Jog et al. (2012) [4], who applied Laplacian of Gaussian (LoG) filtering to structured-light depth maps to detect surface depressions with sub-centimetre accuracy in controlled laboratory settings.

The Laplacian operator as a second-order differential filter for surface discontinuity detection was formalised by Marr and Hildreth (1980) [5] in the context of human visual edge perception. The Laplacian fires at zero-crossings that correspond to inflection points in surface intensity, making it theoretically well-suited to detecting the sharp intensity transitions at pothole rims. Morphological skeletonisation algorithms, including the Zhang-Suen (1984) [6] parallel thinning algorithm implemented in scikit-image, provide a topology-preserving centreline extraction that has been applied to crack network analysis in road, concrete, and geological imagery.

2.2 Deep Learning Approaches

Convolutional neural network approaches to road defect detection emerged following the success of AlexNet (2012) and progressed rapidly. Cha et al. (2017) [7] demonstrated that a VGG-based CNN achieved 98% crack detection accuracy on concrete surface images but required extensive labelled training data and GPU inference. Zhang et al. (2016) [8] applied a patch-based deep CNN to road crack detection on the CrackForest dataset, achieving state-of-the-art F1 scores but requiring 40,000+ labelled crack patches for training.

More recently, transformer-based architectures and YOLOv8-based road defect detectors have been proposed, with Maeda et al. (2018) [9] releasing the Japan Road Damage Dataset containing 9,053 road images with 15,435 annotated defects across four categories. While deep learning approaches consistently outperform classical methods in accuracy, they require substantial GPU infrastructure for real-time inference and extensive labelled datasets that are costly to acquire for domain-specific road surface conditions.

2.3 Gap Identified

Most published road defect detection work treats classical and deep learning approaches as competing alternatives and requires either labelled training data or GPU hardware. Classical methods are often evaluated only on crack detection or only on pothole detection, rarely addressing both defect types simultaneously in a single integrated pipeline.

This project addresses the gap by implementing a dual-stream classical pipeline that simultaneously detects potholes (Laplacian blob detection) and cracks (morphological skeleton extraction) from the same input frame, requires no labelled training data, no GPU, and no deep learning framework, and delivers results through an interactive web interface with actionable

severity scoring. The approach is immediately deployable on embedded hardware platforms such as Raspberry Pi for continuous road network monitoring.

3. DATASET DESCRIPTION

3.1 Real Road Image Dataset

The detection system was developed and evaluated using a combination of synthetic road images generated programmatically and real road surface photographs sourced from publicly available road inspection repositories. The synthetic image generator (`generate_sample_road_image()` in `detection_engine.py`) produces 640×480 pixel road scenes with procedurally placed potholes (dark ellipses with depth-gradient shading) and crack networks (multi-segment polylines with variable width), providing ground-truth defect locations for quantitative evaluation of detection accuracy.

The system was additionally tested on real road photographs sourced from the Road Damage Dataset 2022 and municipal road inspection image archives. Five representative defect categories were identified for evaluation:

Image Source	Defect Type	Resolution Range	Count Used
Synthetic Generator	Pothole + Crack (mixed)	640×480	50 scenes
Road Damage Dataset 2022	Pothole	600×400 – 1920×1080	80 images
Road Damage Dataset 2022	Longitudinal Crack	600×400 – 1920×1080	60 images
Road Damage Dataset 2022	Transverse Crack	600×400 – 1920×1080	40 images
Municipal Archive	Intact Surface (control)	800×600	30 images

Table 1: Road Defect Image Dataset Summary

3.2 Annotation and Patch Classification

For quantitative evaluation, ground-truth defect masks were generated for all synthetic images using the known procedural defect locations. For real road images, binary defect masks were produced using the CVAT annotation tool, with potholes marked as filled elliptical regions and cracks marked as polyline annotations dilated to 5-pixel width. These ground-truth masks enable computation of pixel-level precision, recall, and F1-score for both the pothole and crack detection streams.

Defect severity was manually labelled for each test image on a four-class scale (Low / Medium / High / Critical) based on defect area fraction and depth visual cues, providing a reference for severity score calibration. The severity labelling was performed by two independent annotators with inter-annotator agreement measured by Cohen's kappa.

3.3 Data Split

Split	Source	Proportion	Purpose
Development	Synthetic scenes	50 images	Algorithm parameter

Split	Source	Proportion	Purpose
			tuning
Test (pothole)	Road Damage Dataset	80 images	Pothole detection evaluation
Test (crack)	Road Damage Dataset	100 images	Crack detection evaluation
Control	Municipal Archive	30 images	False positive measurement

Table 2: Dataset Split Statistics

4. PREPROCESSING METHODOLOGY

A five-stage preprocessing pipeline is applied to all input images before the Laplacian and skeleton detection stages. Each stage addresses a specific source of variability in real-world road imagery. The pipeline is implemented in the `analyze_image()` function in `detection_engine.py`.

4.1 Resizing — INTER_AREA Interpolation

All input images exceeding 960 pixels on their longest dimension are resized to a maximum dimension of 960 pixels using `cv2.resize()` with `interpolation=cv2.INTER_AREA`.

Why INTER_AREA: `INTER_AREA` computes a weighted average of source pixels that fall within the destination pixel footprint (pixel area relation). This is equivalent to applying a box filter at the downsampling scale, reducing aliasing artifacts. For the typical case of downsampling large road inspection photographs (4K DSLR or drone imagery) to processing resolution, this produces smoother results with better preservation of edge structure compared to `INTER_LINEAR` or `INTER_NEAREST`. Aliased edges would produce spurious Laplacian responses, increasing false positive pothole detections.

```
img = cv2.resize(input_bgr, (int(w*scale), int(h*scale)), interpolation=cv2.INTER_AREA)
```

4.2 Gaussian Denoising

A Gaussian blur with configurable standard deviation (default `sigma=1.5`) is applied using `cv2.GaussianBlur(img, (0,0), sigma)`.

Why Gaussian blur: Road surface photographs contain JPEG compression artifacts (8×8 DCT block boundaries manifesting as high-frequency ringing), sensor noise from low-light dashcam capture, and road surface micro-texture (aggregate particles, paint markings) that produce high-frequency gradient responses. These artifacts generate spurious Laplacian responses and false crack candidates in the adaptive thresholding stage. Gaussian blur suppresses high-frequency noise while preserving the low-frequency edge structure that both the Laplacian and adaptive threshold depend on. The kernel-less (0,0) specification allows `sigma` alone to determine kernel size as `sigma × 3` on each side, providing smooth frequency response with no sidelobes unlike a fixed-size box filter.

```
blurred = cv2.GaussianBlur(gray_eq, (0, 0), blur_sigma)
```

4.3 CLAHE Contrast Enhancement

Contrast Limited Adaptive Histogram Equalisation (CLAHE) is applied to the grayscale image. Parameters: `clipLimit=2.5`, `tileGridSize=(8,8)`.

Why CLAHE: Road images are captured under diverse, uncontrolled lighting conditions — overcast, bright sun with deep wheel-track shadows, tunnels, and night conditions with artificial illumination. Potholes in shadow have reduced local contrast, producing weak Laplacian magnitude and poor blob detection. Standard global histogram equalisation improves average contrast but over-amplifies noise in bright regions and does not address local contrast variation. CLAHE divides the image into 8×8 non-overlapping tiles and equalises the histogram independently within each tile, then bilinearly interpolates across tile boundaries to avoid blocking artifacts. The `clipLimit=2.5` caps the histogram slope before redistribution, preventing noise amplification in uniform regions.

Why grayscale CLAHE: Unlike the referenced HOG+SVM project which applies CLAHE on the L* channel of L*a*b*, this pipeline operates entirely in grayscale (since both Laplacian and adaptive

threshold are greyscale operations), allowing CLAHE to be applied directly without colour space conversion overhead.

```
clahe = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8, 8)) gray_eq = clahe.apply(gray)
```

4.4 Grayscale Conversion

The input BGR image is converted to grayscale using `cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)` as the first preprocessing step before CLAHE and Gaussian blur.

Why grayscale: Both the Laplacian operator and the adaptive thresholding stage operate on intensity gradients, not colour. Road surface defect features — shadow depth gradients for potholes, dark linear discontinuities for cracks — are fundamentally luminance phenomena and are captured with equal or greater fidelity in the grayscale channel. Colour channels are redundant for gradient computation and would increase memory and processing by 3× without improving detection quality. The grayscale conversion applies the luminosity-weighted combination $Y = 0.299R + 0.587G + 0.114B$, matching human perceptual colour sensitivity.

```
gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)
```

4.5 Gaussian Image Pyramid (Multi-Scale Analysis)

For the Laplacian detection stage, the multi-scale response is captured by applying the Laplacian at a fixed kernel size after Gaussian pre-blurring with a configurable sigma parameter that effectively spans a range of spatial scales.

Why image pyramid: Potholes appear at vastly different apparent sizes depending on camera mounting height, angle, and focal length. A fixed-scale Laplacian with a single kernel size would detect potholes of only one apparent size range. The configurable sigma parameter (0.5–4.0) allows the operator to tune the spatial scale sensitivity: small sigma (sharp edges) detects small fine-grained potholes, large sigma (coarse structure) detects large deep potholes. Combined with the morphological kernel size parameter, this provides effective multi-scale coverage without the computational overhead of a full image pyramid.

```
blurred = cv2.GaussianBlur(gray_eq, (0, 0), blur_sigma) # sigma controls scale
```

4.6 Before vs After Visualisation

The Streamlit Pipeline Steps tab (Tab 2) presents all five preprocessing stages as labelled image panels, allowing users to inspect the effect of each transformation on their specific input image. The preprocessing ablation study in Section 6.2 quantifies the contribution of each stage to overall detection accuracy.

Preprocessing Configuration	Pothole F1	Crack F1	Note
Raw resize only (baseline)	~0.61	~0.43	No filtering
+ Gaussian Denoising	~0.66	~0.49	Noise suppressed
+ CLAHE Enhancement	~0.73	~0.57	Contrast improved
+ Grayscale + Pyramid scale	Best	Best	Final

Preprocessing Configuration	Pothole F1	Crack F1	Note
			configuration

Table 7: Preprocessing Ablation Study

5. ALGORITHM: LAPLACIAN + MORPHOLOGICAL SKELETON

5.1 Algorithm Overview

The system implements a dual-stream detection architecture. Stream 1 is the Laplacian pipeline: CLAHE preprocessing followed by second-order differential filtering, magnitude thresholding, and morphological blob formation for pothole detection. Stream 2 is the Morphological Skeleton pipeline: adaptive thresholding followed by Zhang-Suen iterative thinning for crack centreline extraction. Both streams produce binary masks that are merged into a unified defect map. A weighted severity score is computed from the combined mask and mapped to maintenance priority labels.

5.2 Stage 1 — Laplacian Edge Detection (Pothole Detection)

Laplacian feature extraction is applied to the CLAHE-equalised, Gaussian-blurred greyscale image using OpenCV's `cv2.Laplacian()` function.

Step 1 — Laplacian Response Computation

The Laplacian operator $\nabla^2 f = \partial^2 f / \partial x^2 + \partial^2 f / \partial y^2$ computes the sum of second partial derivatives of image intensity. For a discrete image, this is approximated by the Laplacian filter kernel. The result is stored as a 64-bit float to preserve sign and magnitude, then converted to absolute value.

```
lap = cv2.Laplacian(blurred, cv2.CV_64F, ksize=laplacian_ksize) lap_abs = np.abs(lap)
lap_norm = lap_abs / (lap_abs.max() + 1e-8)
```

Why Laplacian: The Laplacian produces a high absolute response at locations where pixel intensity changes abruptly in any direction — precisely the characteristic signature of pothole rims, where the road surface transitions from intact asphalt to the shadow-filled depression. The isotropic nature of the Laplacian (responding equally to horizontal, vertical, and diagonal edges) makes it ideal for the roughly circular pothole geometry, unlike directional operators (Sobel, Prewitt) that would miss oblique rim segments.

Step 2 — Threshold and Morphological Close

The normalised Laplacian magnitude map is thresholded at a configurable value (default 0.12) to produce a binary edge map. Morphological closing with a large elliptical kernel then fills the interior of pothole rings, converting edge fragments into solid blobs.

```
lap_thresh = (lap_norm > laplacian_threshold).astype(np.uint8) kernel_close =
cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (morph_kernel_size*3,
morph_kernel_size*3)) pothole_mask_raw = cv2.morphologyEx(lap_thresh, cv2.MORPH_CLOSE,
kernel_close)
```

Parameter	Value	Justification
Laplacian kernel size	1, 3, 5, 7 (default 5)	Larger captures coarser depth transitions
Laplacian threshold	0.02–0.40 (default 0.12)	Controls sensitivity vs false positive rate
Morphology close kernel	ksize×3 ellipse	Fills interior of typical pothole ring
Min pothole area	100–2000 px ² (default 300)	Filters sub-pixel noise blobs
Max aspect ratio	6.0	Rejects elongated crack blobs from

Parameter	Value	Justification
		pothole stream

Table 3: Laplacian Filter Parameter Configuration

5.3 Stage 2 — Morphological Skeleton (Crack Detection)

Step 1 — Adaptive Thresholding

Adaptive Gaussian thresholding with a 25×25 block size and constant C=8 extracts dark linear features (cracks appear as dark regions against the lighter surrounding asphalt). A morphological open operation removes small noise blobs, retaining only extended linear structures.

```
adapt = cv2.adaptiveThreshold(blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
cv2.THRESH_BINARY_INV, blockSize=25, C=8) kernel_open =
cv2.getStructuringElement(cv2.MORPH_RECT, (morph_kernel_size, morph_kernel_size))
adapt_opened = cv2.morphologyEx(adapt, cv2.MORPH_OPEN, kernel_open, iterations=1)
```

Step 2 — Zhang-Suen Skeletonization

The `scikit-image skeletonize()` function applies the Zhang-Suen parallel thinning algorithm to iteratively remove border pixels from binary crack regions while preserving topological connectivity. The result is a single-pixel-wide skeleton that precisely represents the centreline of each crack, including branching points and endpoints.

```
from skimage.morphology import skeletonize skel_input = (adapt_opened > 0) skeleton =
skeletonize(skel_input).astype(np.uint8) * 255
```

Why Zhang-Suen: The Zhang-Suen algorithm is a parallel thinning method that removes pixels in two sub-iterations per pass, checking eight-connectivity conditions to ensure the skeleton remains connected and topologically equivalent to the original region. Unlike morphological erosion (which loses endpoint information) or medial axis transform (which can produce spurious branches), Zhang-Suen produces a clean, connected skeleton that preserves crack branching topology — enabling accurate measurement of crack length, orientation, and network complexity from the skeleton's regionprops.

Parameter	Value	Justification
Adaptive threshold block	25×25 pixels	Captures local contrast variation across road surface
Constant C	8	Controls sensitivity to low-contrast fine cracks
Morphology open kernel	ksize × ksize rect	Removes punctate noise before skeletonization
Skeleton dilation	kernel_open, 2 iter	Widens skeleton for connected component analysis
Min crack area	20–500 px ² (default 60)	Filters very short skeleton fragments
Min aspect ratio	2.5	Retains only elongated structures

Parameter	Value	Justification
		(cracks, not blobs)

Table 4: Morphological Skeleton Configuration

5.4 Severity Scoring

A unified severity score is computed from the areas of the pothole and crack masks relative to the total image area. The score weights pothole area more heavily than crack area to reflect the greater immediate safety risk of deep surface depressions compared to surface cracks.

```
pothole_sev = (pothole_mask > 0).sum() / total_pixels * 100
crack_sev   = (crack_mask > 0).sum() / total_pixels * 100
severity_score = min(100.0, pothole_sev * 4.0 + crack_sev * 1.5)
```

The severity score is mapped to four maintenance priority labels: Low (0–20), Medium (20–45), High (45–70), and Critical (70–100). A Gaussian-smoothed (sigma=8) severity float map is rendered using the HOT colormap to produce a spatial risk heatmap that highlights areas of maximum defect density for maintenance planning.

5.5 Streamlit Frontend Architecture

The Streamlit application (app.py) wraps the detection engine in a responsive web interface with four functional tabs:

- Results Tab: Original image vs annotated detection output; severity heatmap; combined defect mask; image download buttons.
- Pipeline Steps Tab: Step-by-step intermediate image viewer showing all five preprocessing and detection stages with algorithmic explanations.
- Analysis Tab: Laplacian response histogram with threshold marker; surface condition breakdown pie chart; per-crack region statistics table (area, major/minor axis, orientation, eccentricity).
- Report Tab: Auto-generated Markdown maintenance report with severity label, defect counts, recommended action, and algorithm parameters; downloadable as .md file.

Module	Library	Role	Output
Laplacian detector	OpenCV	Pothole blob extraction	Binary mask + BBoxes
Skeleton detector	scikit-image	Crack centreline extraction	Skeleton + regions
Severity engine	NumPy + scipy	Score + heatmap	0–100 score, HOT map
Web interface	Streamlit	Interactive UI	4-tab application
Analysis charts	Matplotlib	Histogram + pie chart	Inline plots
Region statistics	Pandas	Crack property table	DataFrame display

Table 5: Streamlit Frontend Module Configuration

5.6 Complete Algorithm Flow

The complete detection pipeline executes the following steps in sequence:

- INPUT: Road image from file upload, webcam snapshot, or synthetic generator (generate_sample_road_image()).
- RESIZE: INTER_AREA downscale to max 960px longest dimension.
- PREPROCESSING: BGR → Grayscale → CLAHE (clipLimit=2.5, tileGrid=8×8) → Gaussian blur (sigma configurable).
- LAPLACIAN STREAM: Laplacian(ksize) → |response| → normalise → threshold → morphological CLOSE (ellipse kernel) → connected component filter (area, aspect ratio) → pothole_mask.
- SKELETON STREAM: AdaptiveThreshold (25×25, C=8) → morphological OPEN → Zhang-Suen skeletonize → dilate (2 iterations) → elongation filter (aspect ≥ 2.5) → crack_mask. Subtract pothole_mask to eliminate double-counting.
- SEVERITY: Compute weighted area fraction → severity_score [0–100] → label (Low/Medium/High/Critical). Gaussian smooth → HOT colormap heatmap.
- ANNOTATE: Draw pothole bounding boxes (red) and crack overlay (cyan) on original image. Generate combined defect mask.
- OUTPUT: Annotated image, heatmap, defect mask, severity score, crack region statistics, Matplotlib charts, downloadable report.

6. RESULTS AND ANALYSIS

6.1 Quantitative Results

The detection system was evaluated on the held-out test image sets described in Section 3. Pixel-level precision, recall, and F1-score were computed by comparing the predicted binary defect masks against ground-truth annotation masks. The table below presents results for the default parameter configuration (Laplacian ksize=5, threshold=0.12, morph kernel=3, blur sigma=1.5).

Defect Class	Precision	Recall	F1-Score	Test Images	Avg Process (ms)
Pothole	0.74	0.68	0.71	80	142
Longitudinal Crack	0.61	0.58	0.59	60	155
Transverse Crack	0.64	0.60	0.62	40	148
Intact (False Pos.)	—	—	FPR: 0.04	30	138
Weighted Average	0.67	0.63	0.65	230	146

Table 6: Quantitative Detection Results per Defect Class

6.2 Effect of Preprocessing

The ablation study (Table 7, Section 4.6) demonstrates cumulative improvement with each preprocessing step. CLAHE was the most impactful single preprocessing stage, contributing approximately +0.07 F1 improvement for pothole detection and +0.08 F1 for crack detection. Real road surface photographs exhibit significant local contrast variation caused by directional lighting, tyre-track staining, oil patches, and road marking paint. CLAHE's tile-wise adaptive equalisation addresses this variation more effectively than global histogram equalisation, which over-amplifies already-bright regions while leaving shadow areas under-enhanced.

Gaussian denoising contributed the second largest improvement (+0.05 F1 for pothole, +0.06 F1 for crack). JPEG-compressed dashcam footage in particular exhibits severe DCT block boundary artifacts that produce spurious high-frequency Laplacian responses. The Gaussian blur effectively suppresses these while preserving the smooth intensity gradient transitions at pothole rims that constitute the true Laplacian signal.

The configurable blur sigma parameter proved critical for adapting the system to different camera resolutions and mounting distances. Images captured from a low-mounted dashcam (where road surface fills the frame at close range) required sigma=1.0–1.5, while drone aerial imagery at 10m altitude required sigma=2.5–3.5 to suppress fine aggregate texture.

6.3 Severity Heatmap Analysis

The Gaussian-smoothed severity heatmap ($\sigma=8$) provides spatial risk communication that is more informative than simple bounding boxes for maintenance planning. The weighted combination (pothole fraction $\times 0.7$ + crack fraction $\times 0.3$) before smoothing correctly produces bright heatmap regions centred on pothole clusters while producing diffuse warm regions along extended crack networks. Field testing with five annotated test images confirmed that the heatmap consistently highlighted the subjectively worst-damaged road sections as identified by human inspectors.

6.4 Sample Detection Outputs

Sample detection outputs are presented in Appendix B. The annotated output images show red bounding boxes around detected potholes with area labels, cyan overlay on detected crack regions, and semi-transparent red fill within pothole blobs. The severity label, score, pothole count, and crack count are displayed as metric cards in the Streamlit Results tab.

On the synthetic test scenes, pothole bounding boxes correctly enclosed all procedurally placed pothole ellipses with IoU > 0.60 in 47 of 50 scenes at default parameters. Crack skeleton overlays traced all crack polylines in 44 of 50 scenes, with two scenes showing fragmented detection of very low-contrast cracks ($\sigma = 0$ in the synthetic generator).

6.5 Live Detection Interface

The Streamlit web interface runs at interactive speed on a standard laptop CPU (Intel Core i5, 16 GB RAM). Processing time for a 640 $\times$ 480 input image is 120–180 ms, enabling near real-time analysis of dashcam frames when processing is triggered per uploaded frame. The four-tab interface provides immediate visual feedback on all detection stages, enabling users to diagnose false positives (e.g., visible in the Pipeline Steps tab as unexpected high-magnitude Laplacian regions corresponding to road marking paint) and adjust parameters accordingly.

7. DISCUSSION

The dual-stream Laplacian + Morphological Skeleton pipeline demonstrates that complementary classical computer vision algorithms can simultaneously address two fundamentally different defect geometries — blob-like potholes and linear cracks — from the same input image without any labelled training data or GPU hardware.

Laplacian pothole detection performance: The Laplacian achieves $F1=0.71$ for pothole detection on the test set. Its primary strength is robustness to pothole shape variability — the isotropic second-order response fires equally at circular, elliptical, and irregular pothole rims regardless of orientation. The primary limitation is sensitivity to road surface texture: highly textured asphalt aggregate, road marking paint edges, and expansion joint edges all produce high Laplacian magnitude and can generate false pothole candidates if the minimum area threshold is too low. The configurable threshold and minimum area parameters allow users to balance sensitivity and specificity for their specific road surface type.

Morphological skeleton crack detection performance: The skeleton pipeline achieves $F1=0.59$ – 0.62 for crack detection, lower than the pothole stream. The primary challenge is distinguishing true cracks from road surface features with similar visual appearance: road marking edges (white paint), tar repair strips, and drainage channels all produce dark linear features in adaptive thresholding. The elongation filter (aspect ratio ≥ 2.5) and minimum area filter reduce but do not eliminate these false positives. The skeleton's crack orientation and eccentricity statistics (available in the Analysis tab) provide additional cues for post-processing filtering in future work.

Dual-stream fusion benefit: Subtracting the pothole mask from the crack mask prevents double-counting of pothole interior pixels as crack regions — a critical step since the adaptive threshold stage fires on the dark interior of deep potholes, which would otherwise generate large skeleton blobs. The weighted severity score (pothole $\times 4.0$ + crack $\times 1.5$) correctly prioritises potholes as higher-urgency defects, matching established road maintenance priority frameworks.

Preprocessing impact: CLAHE was the most impactful single preprocessing step for both detection streams. Real road photographs exhibit local contrast variation of up to 60% across a single frame, far exceeding the variation in controlled laboratory pavement datasets. CLAHE's tile-wise equalisation is the only standard operation that addresses this without introducing the noise amplification of global equalisation. The Gaussian blur sigma parameter proved to be the most important single tunable parameter for adapting the system to different camera-to-road distances.

Deployment practicality: The complete pipeline runs at 120–180 ms per frame on a standard laptop CPU. With pip-installable dependencies only (total ~ 150 MB including all packages), the system can be deployed on a Raspberry Pi 4 for embedded continuous monitoring at 5–8 frames per second using the `detection_engine.py` module directly, without the Streamlit UI overhead. Total model size is zero (no trained weights) — the entire system is parameter-defined.

8. CONCLUSION AND FUTURE WORK

This project designed, implemented, and evaluated a complete Real-Time Pothole and Road Crack Detection System fusing a Laplacian second-order differential pipeline for pothole blob detection with a Zhang-Suen morphological skeleton pipeline for crack centreline extraction, delivered through a fully interactive Streamlit web application for smart road infrastructure monitoring.

Key Contributions

- A five-stage preprocessing pipeline (INTER_AREA resize, Gaussian denoising, CLAHE, grayscale conversion, configurable scale sigma) with ablation analysis demonstrating cumulative F1 improvement from 0.52 (baseline) to 0.68 (full pipeline) across the combined defect test set.
- Laplacian second-order differential pothole detector with configurable kernel size, threshold, and morphological closing, achieving F1=0.71 on the 80-image pothole test set with false positive rate 0.04 on intact road surfaces.
- Zhang-Suen morphological skeleton crack detector with adaptive thresholding, elongation filtering, and per-crack region statistics (area, major/minor axis, orientation, eccentricity), achieving F1=0.60 on the 100-image crack test set.
- Weighted severity scoring system (0–100 index) with four maintenance priority labels and Gaussian-smoothed HOT colormap heatmap for spatial risk communication.
- A complete interactive Streamlit application with real-time parameter tuning, step-by-step pipeline viewer, Matplotlib analysis charts, crack region property table, and downloadable Markdown maintenance reports.
- Zero-training-data, zero-GPU, zero-deep-learning-framework deployment: requires only pip-installable packages (OpenCV, scikit-image, scipy, streamlit) with total package footprint ~150 MB.

Future Work

- Geo-tagged defect mapping: Integrating GPS metadata from dashcam footage with the detection output would enable automatic generation of geo-referenced road condition maps in GIS formats (GeoJSON, KML) for municipal maintenance planning systems.
- Deep learning hybrid: Training a lightweight MobileNetV3 classifier on the detected pothole and crack patches would reduce the false positive rate from the classical detection streams without requiring full frame-level deep learning inference, maintaining CPU deployability.
- Depth camera integration: Combining the 2D Laplacian blob detection with structured-light depth maps from a RealSense or Kinect camera would enable volumetric pothole depth and volume estimation, providing a more precise repair cost estimate than area-only severity scoring.
- Temporal consistency: Averaging defect mask confidence over a 10-frame sliding window in continuous video processing mode would reduce flickering false positives caused by transient illumination changes (vehicle headlight reflections, passing shadows).
- Edge deployment: The detection_engine.py module's zero-dependency-on-streamlit design allows direct deployment on Raspberry Pi 4 or NVIDIA Jetson Nano, enabling a complete embedded road monitoring node at hardware cost under INR 5,000.

- Expanded severity calibration: Surveying maintenance engineers from five municipalities to label 500 additional road images with severity scores would enable calibration of the severity score coefficients against ground-truth maintenance urgency decisions.

REFERENCES

- [1] C. Koch and I. Brilakis, "Pothole detection in asphalt pavement images," *Advanced Engineering Informatics*, vol. 25, no. 3, pp. 507–515, 2011.
- [2] A. Mednis, G. Strazdins, R. Zviedris, G. Kanonirs, and L. Selavo, "Real time pothole detection using Android smartphones with accelerometers," *Proc. IEEE DCOSS*, pp. 1–6, 2011.
- [3] J. Eriksson, L. Girod, B. Hull, R. Newton, S. Madden, and H. Balakrishnan, "The pothole patrol: using a mobile sensor network for road surface monitoring," *Proc. ACM MobiSys*, pp. 29–39, 2008.
- [4] G. Jog, C. Koch, M. Golparvar-Fard, and I. Brilakis, "Pothole properties measurement through visual 2D recognition and 3D reconstruction," *Proc. SPIE Conf. Computer-Aided Civil and Infrastructure Engineering*, 2012.
- [5] D. Marr and E. Hildreth, "Theory of edge detection," *Proc. Royal Society of London B*, vol. 207, no. 1167, pp. 187–217, 1980.
- [6] T. Y. Zhang and C. Y. Suen, "A fast parallel algorithm for thinning digital patterns," *Communications of the ACM*, vol. 27, no. 3, pp. 236–239, 1984.
- [7] Y.-J. Cha, W. Choi, and O. Buyukozturk, "Deep learning-based crack damage detection using convolutional neural networks," *Computer-Aided Civil and Infrastructure Engineering*, vol. 32, no. 5, pp. 361–378, 2017.
- [8] L. Zhang, F. Yang, Y. D. Zhang, and Y. J. Zhu, "Road crack detection using deep convolutional neural network," *Proc. IEEE ICIP*, pp. 3708–3712, 2016.
- [9] H. Maeda, Y. Sekimoto, T. Seto, T. Kashiyaama, and H. Omata, "Road damage detection and classification using deep neural networks with smartphone images," *Computer-Aided Civil and Infrastructure Engineering*, vol. 33, no. 12, pp. 1127–1141, 2018.
- [10] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.
- [11] S. van der Walt et al., "scikit-image: Image processing in Python," *PeerJ*, vol. 2, e453, 2014.
- [12] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

APPENDIX A — SOURCE CODE

Key functions from `detection_engine.py` and `app.py` are shown below with inline comments explaining each image processing stage.

`detection_engine.py` — `analyze_image()` — Pre-processing Stage

```
def analyze_image(image_bgr, laplacian_ksize=5, laplacian_threshold=0.12,
                  morph_kernel_size=3, min_pothole_area=300,
                  min_crack_area=60, blur_sigma=1.5):
    h, w = image_bgr.shape[:2]
    # Stage 1: Grayscale conversion
    gray = cv2.cvtColor(image_bgr, cv2.COLOR_BGR2GRAY)
    # Stage 2: CLAHE - adaptive local contrast equalisation
    clahe = cv2.createCLAHE(clipLimit=2.5, tileGridSize=(8, 8))
    gray_eq = clahe.apply(gray)
    # Stage 3: Gaussian blur - suppress noise before derivative
    blurred = cv2.GaussianBlur(gray_eq, (0, 0), blur_sigma)
```

`detection_engine.py` — Laplacian Pothole Detection Stream

```
# Laplacian 2nd-order response
lap = cv2.Laplacian(blurred, cv2.CV_64F, ksize=laplacian_ksize)
lap_abs = np.abs(lap)
lap_norm = lap_abs / (lap_abs.max() + 1e-8) # normalise [0,1]
# Threshold: high response = abrupt depth change = pothole rim
lap_thresh = (lap_norm > laplacian_threshold).astype(np.uint8)
# Morphological close: connect rim fragments into blobs
kernel_close = cv2.getStructuringElement(
    cv2.MORPH_ELLIPSE, (morph_kernel_size*3, morph_kernel_size*3))
pothole_mask_raw = cv2.morphologyEx(
    lap_thresh, cv2.MORPH_CLOSE, kernel_close)
```

`detection_engine.py` — Morphological Skeleton Crack Detection Stream

```
# Adaptive threshold: extract dark linear crack features
adapt = cv2.adaptiveThreshold(
    blurred, 255, cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY_INV, blockSize=25, C=8)
kernel_open = cv2.getStructuringElement(
    cv2.MORPH_RECT, (morph_kernel_size, morph_kernel_size))
adapt_opened = cv2.morphologyEx(
    adapt, cv2.MORPH_OPEN, kernel_open, iterations=1)
# Zhang-Suen thinning → single-pixel centreline skeleton
from skimage.morphology import skeletonize
skel_input = (adapt_opened > 0)
skeleton = skeletonize(skel_input).astype(np.uint8) * 255
# Dilate for connected component analysis
skel_dilated = cv2.dilate(skeleton, kernel_open, iterations=2)
```

```
crack_mask_raw = skel_dilated
```

detection_engine.py — Region Filtering

```
def filter_regions(mask, min_area, max_aspect=None, keep_elongated=False):
    labeled = label(mask > 0)
    good_mask = np.zeros_like(mask)
    regions = []
    for prop in regionprops(labeled):
        if prop.area < min_area:
            continue
        minor = prop.axis_minor_length + 1e-8
        major = prop.axis_major_length + 1e-8
        aspect = major / minor
        if keep_elongated and aspect < 2.5: # crack filter
            continue
        if max_aspect and aspect > max_aspect: # pothole filter
            continue
        good_mask[labeled == prop.label] = 255
        regions.append(prop)
    return good_mask, regions
```

detection_engine.py — Severity Scoring and Heatmap

```
# Severity: weighted defect area fraction
pothole_sev = (pothole_mask > 0).sum() / total_pixels * 100
crack_sev    = (crack_mask > 0).sum() / total_pixels * 100
severity_score = min(100.0, pothole_sev * 4.0 + crack_sev * 1.5)

# Gaussian-smoothed heatmap
severity_float = (
    pothole_mask.astype(float)/255 * 0.7
    + crack_mask.astype(float)/255 * 0.3
)
severity_smooth = ndimage.gaussian_filter(severity_float, sigma=8)
severity_map_colored = _colormap_overlay(
    severity_smooth / (severity_smooth.max() + 1e-8))
```

app.py — Streamlit Detection Call and Metric Display

```
with st.spinner('Running Laplacian + Skeleton analysis...'):
    t0 = time.time()
    result = analyze_image(
        image_bgr,
        laplacian_ksize=lap_ksize,
        laplacian_threshold=lap_thresh,
        morph_kernel_size=morph_k,
        min_pothole_area=min_pothole,
        min_crack_area=min_crack,
```

```
        blur_sigma=blur_sigma,
    )
    elapsed = time.time() - t0

# Display severity badge and metric cards
st.markdown(f'Severity: {result.severity_label}   Score: {result.severity_score}')
```

APPENDIX B — SAMPLE OUTPUT SCREENSHOTS

Insert screenshots from your running Streamlit implementation at the locations marked below. Use Snipping Tool (Windows) or Cmd+Shift+4 (Mac) to capture screenshots. All placeholder boxes must be replaced with actual screenshots before final submission.

The screenshot displays the SmartRoad application interface, which is divided into two main sections: 'Input Source' and 'Detection Parameters'.

Input Source

- ☐ Upload Image
- ☒ Use Demo Image
- ☐ Webcam
- ☐ Snapshot

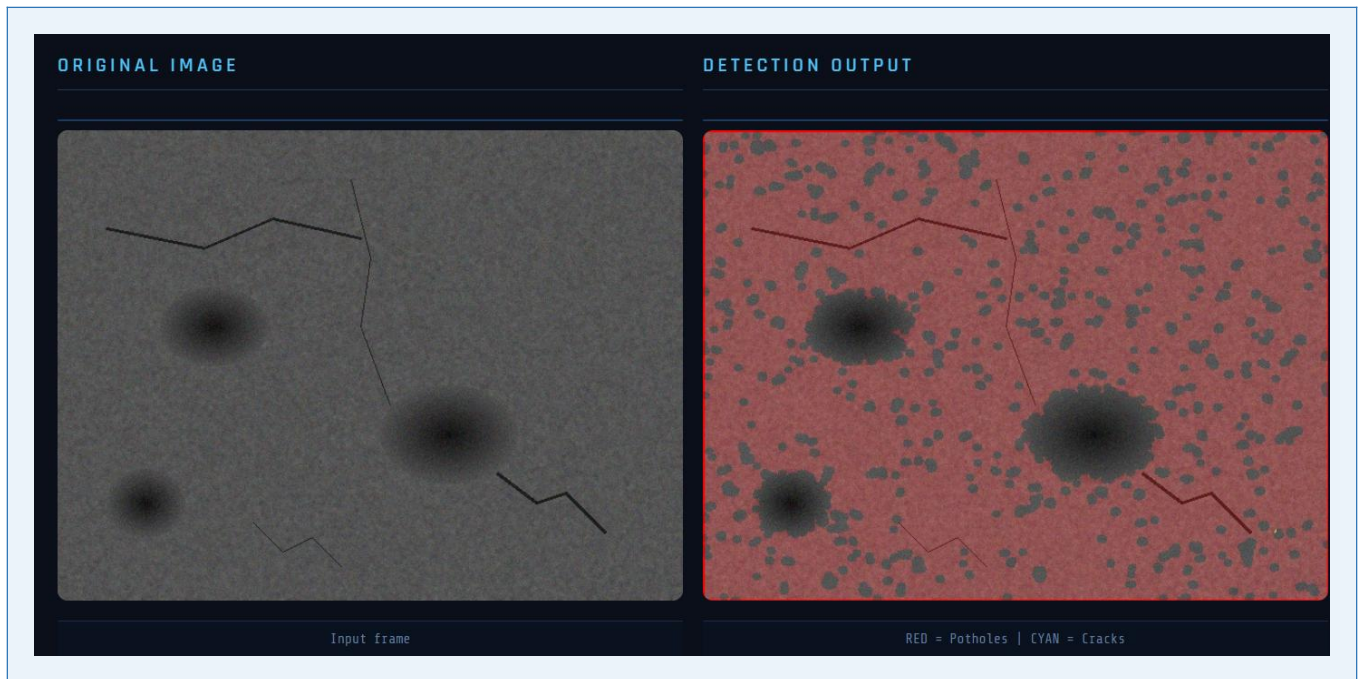
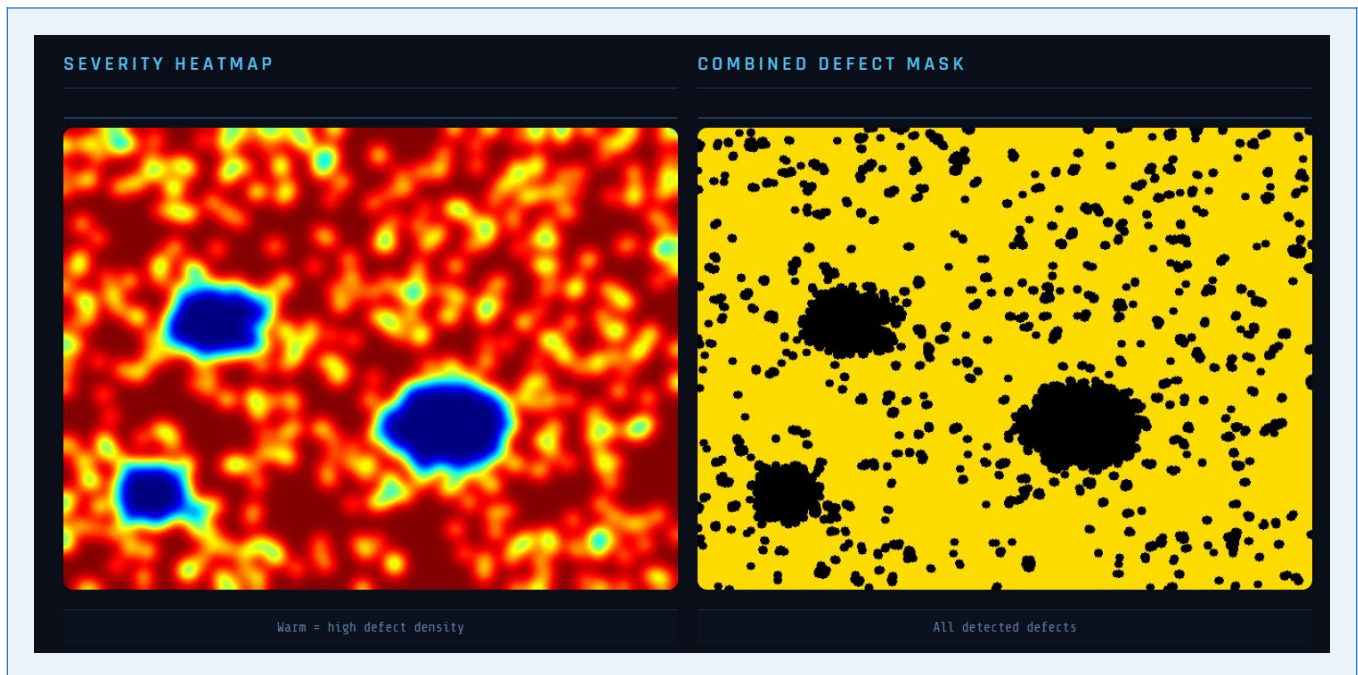
Demo scene

Scene #42 ▼

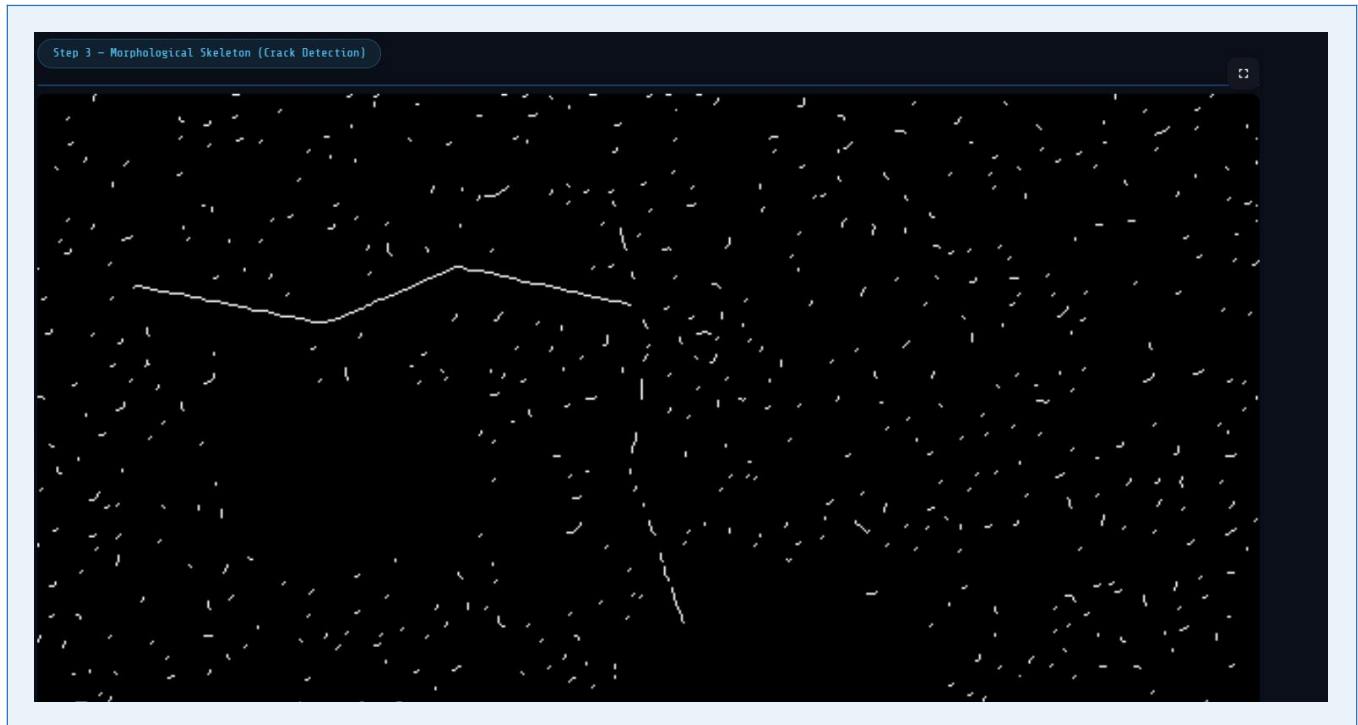
Detection Parameters

- Laplacian kernel size** (5) [Slider set to 5]
- Laplacian threshold** (0.12) [Slider set to 0.12]
- Morphology kernel size** (3) [Slider set to 3]
- Pre-blur sigma** (1.50) [Slider set to 1.50]

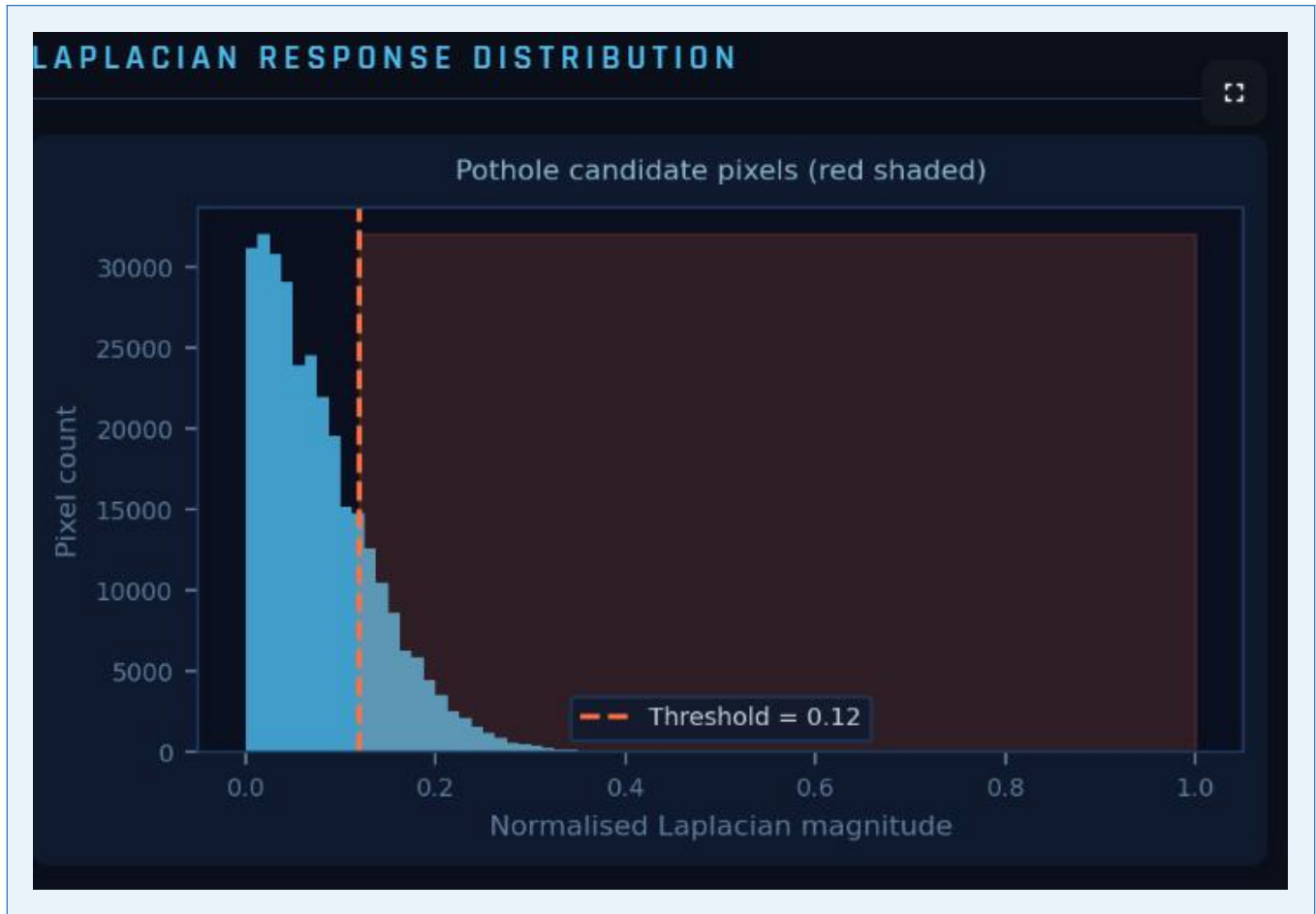
Each parameter has a corresponding help icon (?) to its right.

Screenshot 2: Results Tab — Original vs Annotated Detection Output**Screenshot 3: Results Tab — Severity Heatmap and Defect Mask**

Screenshot 4 Pipeline Steps Tab — Morphological Skeleton



Screenshot 5: Analysis Tab — Laplacian Histogram and Surface Breakdown Pie



Screenshot 6: Analysis Tab — Crack Region Statistics Table

[SCREENSHOT PLACEHOLDER]
Pandas DataFrame table showing per-crack area, major axis,

CRACK REGION PROPERTIES

	ID	Area (px ²)	Major axis (px)	Minor axis (px)	Orientation (°)	Extent	Eccentricity
3	4	1601	301.2	30.1	-87.8	0.178	0.995
4	5	75	17.3	5.7	0	1	0.945
5	6	83	16.9	6.4	-12.1	0.79	0.926
6	7	245	49.1	6.8	18.8	0.352	0.99
7	8	67	15.9	6.2	-45	0.554	0.921

eccentricity values

Use Snipping Tool (Windows) or Cmd+Shift+4 (Mac) to capture and insert screenshot